

Artur Matoso Nery

Desenvolvedor Full Stack

E-mail: arturnery1997@gmail.com

LinkedIn: [linkedin.com/in/artur-matoso-nery-84a4971a9/](https://www.linkedin.com/in/artur-matoso-nery-84a4971a9/)

GitHub: github.com/arturnery

Portfólio: arturnery.vercel.app

RESUMO PROFISSIONAL

Desenvolvedor Full Stack com foco em TypeScript, React/Next.js e Node.js, com experiência prática construindo aplicações completas, da modelagem do banco ao deploy em produção. Como freelancer, entreguei três projetos em produção para o mesmo cliente, trabalhando com type-safety end-to-end (tRPC + Zod), bancos serverless, testes automatizados com Vitest e infraestrutura própria (Vercel, Docker em VM Linux). Busco minha primeira oportunidade como desenvolvedor júnior, para contribuir em times de produto e evoluir em boas práticas, code review e desenvolvimento colaborativo.

EXPERIÊNCIA

Desenvolvedor Full Stack Freelancer (atual)

Cliente recorrente: **Level Cripto**. Três projetos entregues para o mesmo cliente, todos em produção e em evolução contínua: uma aplicação multiusuário de controle financeiro, a landing page da plataforma e um bot de calendário econômico no Discord.

Airdrop Tracker: aplicação multiusuário de controle financeiro, em produção

Demo: lvl-airdrop.vercel.app · Repositório: github.com/arturnery/airdrop-tracker

- Recebi do cliente o problema, não a solução: a comunidade controlava o farming de airdrops em planilhas e não conseguia responder quanto tinha investido nem qual era o retorno real. Levantei os requisitos, modeliei o domínio e escolhi a stack sozinho.
- Escrevi o documento de arquitetura antes de implementar, mapeando as três limitações estruturais da planilha: a conta era texto redigitado a cada linha e não uma entidade, depósito e foto de saldo dividiam a mesma coluna (o que tornava todo somatório incorreto) e um único campo de status significava tanto tarefa pendente quanto airdrop não recebido.
- Resolvi modelando o domínio como livro-razão: aportes, rendimentos, retiradas e taxas são lançamentos datados e o saldo passa a ser resultado calculado, assumindo em troca que toda variação precisa ser lançada.
- Organizei o código em três fronteiras explícitas: leitura isolada em consultas dedicadas, escrita concentrada em Server Actions validadas com Zod e cálculo financeiro em funções puras, sem acesso ao banco.
- Construí em Next.js 16 (App Router, Server Components), TypeScript em modo strict, PostgreSQL serverless (Neon) e Drizzle ORM com migrations versionadas sobre 14 tabelas, com autenticação Auth.js (Credentials + JWT), aprovação manual de cadastro e isolamento de dados por usuário.
- Cobri as funções de cálculo financeiro com 159 testes automatizados em Vitest, viabilizados por elas serem puras.

Level Cripto PRO: landing page full stack em produção

Projeto: www.levelcripto.com.br/ · Repositório: github.com/arturnery/LevelCriptoPRO

- Desenvolvi e publiquei em produção a landing page full stack do Level Cripto PRO, com sistema de captação de leads integrado a banco de dados.
- Migrei a aplicação de uma plataforma proprietária para uma stack open source com infraestrutura própria, reduzindo o custo de hospedagem a zero (Vercel + Neon, free tier).
- Construí o frontend com React 19, TypeScript e Vite, utilizando TailwindCSS e componentes Radix UI (Shadcn/ui), com layout responsivo e seções dinâmicas (countdown, carousel de depoimentos, FAQ).
- Implementei comunicação end-to-end com tRPC, garantindo type-safety entre cliente e servidor sem geração de código, e validação de dados com Zod (schemas únicos compartilhados).
- Desenvolvi backend em Node.js + Express com PostgreSQL serverless (Neon) via Drizzle ORM, incluindo autenticação JWT (jose), tratamento de erros e detecção de e-mails duplicados.
- Escrevi 108 testes automatizados com Vitest cobrindo validações de formulário, máscaras de telefone (BR e internacional), tratamento de erros do Drizzle e fluxo de autenticação.
- Configurei deploy serverless no Vercel: diagnostiquei e resolvi ERR_MODULE_NOT_FOUND em produção pré-compilando o handler tRPC com esbuild como bundle autossuficiente.

EcoBot: bot de calendário econômico para Discord, rodando 24/7 em produção

Repositório: github.com/arturnery/discord-eco-bot

- Desenvolvi um bot em Python que monitora o calendário econômico do Investing.com e entrega automaticamente agenda diária, alertas pré-evento e resultados com análise de surpresa (real vs. previsão) em um canal do Discord, para antecipar janelas de alta volatilidade no mercado.

- Implementei deploy em produção em VM Linux na nuvem (Oracle Cloud Always Free, Ubuntu) com Docker e docker-compose, usando `restart: always` para disponibilidade contínua sem supervisão, com fluxo de atualização via SSH/SCP.
- Diagnosticuei e resolvi um bug que só aparecia em produção: a fonte servia horários conforme o IP do servidor, então tornei o timezone configurável por variável de ambiente, resolvendo sem alterar código.
- Implementei persistência de estado em arquivo com volume Docker para garantir idempotência: alertas e resultados não são reenviados após reinício do container, sem a complexidade de um banco de dados.
- Construí o scraper com BeautifulSoup4 + lxml e retry automático (5 tentativas), além de filtros configuráveis de moeda e impacto via variáveis de ambiente.

PROJETOS

Portfólio pessoal: SPA estático, responsivo e bilíngue

Site: arturnery.vercel.app · Repositório: github.com/arturnery/portfolio

- Construí do zero com React 18 + Vite, sem bibliotecas de UI, cada dependência evitada por decisão registrada no README: CSS Modules com design tokens em custom properties para centralizar o theming, internacionalização PT/EN própria em um objeto mais um hook, e dark/light mode aplicado por script inline antes do React montar, para não piscar o tema errado.
- Separei conteúdo de apresentação, com projetos e perfil em arquivos de dados que a UI só consome, e tratei acessibilidade desde o início: HTML semântico, skip link, foco visível, contraste AA e `prefers-reduced-motion` respeitado.

Venda do Dia: PWA offline-first (TCC do curso de ADS)

Demo: arturnery.github.io/projeto-PEXV-faculdade-descomplica/ · Repositório: github.com/arturnery/projeto-PEXV-faculdade-descomplica

- Desenvolvi um PWA de controle de vendas diárias para uma mercearia de bairro, funcionando offline via Service Worker e com custo de manutenção zero. Representei dinheiro em centavos inteiros para evitar erro de ponto flutuante e isolei a lógica de negócio do DOM para torná-la testável.
- Cobri o sistema com 136 testes automatizados em Vitest, rodando no GitHub Actions a cada push.
- **Stack:** JavaScript, HTML, CSS, PWA, Service Worker, Vitest, GitHub Actions.

Food Explorer: cardápio digital full stack (conclusão da formação Rocketseat)

Demo: foodexplorerrrs.netlify.app · Repositório: github.com/arturnery/food-explorer

- Desenvolvi uma aplicação full stack de cardápio digital para restaurantes: autenticação JWT com acesso por perfil (admin e cliente) em rotas protegidas, CRUD de pratos com upload de imagens e busca por nome ou ingrediente, praticando modelagem relacional e organização cliente/servidor.
- **Stack:** React, Vite, Styled-components, Node.js, Express, SQLite, Knex, JWT.

FORMAÇÃO ACADÊMICA

Pós-graduação em Desenvolvimento Full Stack, Descomplica. Em andamento.

Tecnólogo em Análise e Desenvolvimento de Sistemas, Descomplica. Conclusão: 2025

HABILIDADES TÉCNICAS

- **Linguagens:** TypeScript, JavaScript, Python, SQL
- **Frontend:** React, Next.js (App Router, Server Components), Vite, TailwindCSS, Radix UI / Shadcn, Styled-components, CSS Modules, PWA / Service Worker, acessibilidade (WCAG AA)
- **Backend:** Node.js, Express, tRPC, Server Actions, Zod, JWT (jose), Auth.js, bcrypt, Python (discord.py, BeautifulSoup4)
- **Banco de Dados:** PostgreSQL (Neon serverless), MySQL, SQLite, Drizzle ORM (migrations versionadas), Knex
- **Testes:** Vitest (403 testes escritos entre projetos), jsdom, GitHub Actions
- **DevOps / Deploy:** Docker (docker-compose, volumes, restart policy), Vercel, Oracle Cloud (VM Linux), Git/GitHub, esbuild, SSH/SCP
- **Desenvolvimento assistido por IA:** Claude Code no fluxo diário para implementação, testes e refatoração, com instruções de agente versionadas por projeto, catálogo próprio de 39 padrões de defeito, e revisão crítica de todo código gerado antes do commit
- **Conceitos:** type-safety end-to-end, arquitetura serverless, modelagem de dados, autenticação e isolamento por usuário, deploy em produção

IDIOMAS

Português nativo. Inglês intermediário (leitura e escrita técnica).